© Fernando Barber y Universitat de Valencia. Prohibida la copia o reproducción total o parcial sin previo consentimiento.

Duración: 2h15m **Nombre:**

1. (1.0 ptos) Dadas las siguientes afirmaciones, indica si son verdaderas (V) o falsas (F). Las contestaciones erróneas **restan** la mitad de la puntuación.

a) $lg(2\ lg\sqrt{n^3}) \in O(\sqrt{\sqrt{n}})$ [ ]

b) $2\sqrt{n}\ lgn + 3\sqrt{n^3} \in \omega(n\sqrt{n})$ [ ]

c) $\frac{lg\,n}{n} sen(n!) + 10 \in \Theta(1)$ [ ]

d) $\Theta(n) \subset \Omega(n^2) \cap O(\sqrt{n})$ [ ]

e) $2n\ lg\ n + \frac{1}{2}n^2 \in \Omega\left(n^{\frac{3}{2}}\right)$ [ ]

2. (1.5 ptos)

```
func f(a: vector[1..n] de Natural) dev s: Natural
s ← 0
para i ← 2 hasta n-1
      j ← 1
      mientras j <= i ∧ a[j] != n
            s ← s + a[j]
            j ← j + 1
      fmientras
fpara
```

a) Para n = 4, escribe un vector que corresponda a la instancia mejor y otro que corresponda a la instancia peor.

Instancia mejor:

Instancia peor:

b) Calcula el coste exacto en pasos de los casos mejor y peor.

© Fernando Barber y Universitat de Valencia. Prohibida la copia o reproducción total o parcial sin previo consentimiento.

c) Coste asintótico de la función.

3. (1.0 ptos) Completa la siguiente función recursiva. Ten en cuenta que es posible llamarla con un vector vacío (N = 0).

```
func f(v: Vector[1..N] de Real; i: Natural; x: Real; a:Bool) dev res: Bool

si Cond
    res ← E1
sino
    res ← f(v, E2, x)
    res ← E3

{Post: res = (∀α: i ≤ α ≤ N: v[α] = x) ∧ a}
```

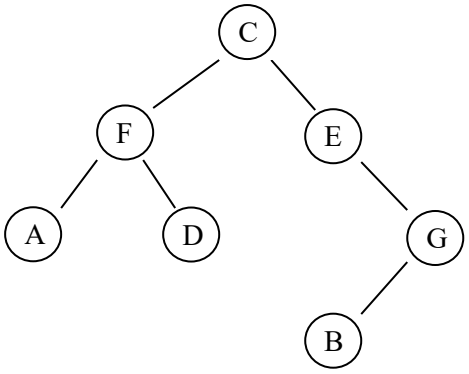
Cond:
E1:
E2:
E3:

4. (1.25 ptos) Calcula el **coste exacto** en pasos y el **coste asintótico** de la siguiente función recursiva. (Resolver en última página)

```
func f(n: Nat, x:Nat) dev s: Nat
si n < 2
      s ← 0
sino
      s ← f(n-2, 2*x)
      para i ← 1 hasta n-1
            s ← s * x + i
      fpara
fsi
```

© Fernando Barber y Universitat de Valencia. Prohibida la copia o reproducción total o parcial sin previo consentimiento.

5. (1.0 pto) Dado el siguiente árbol binario:



a) Recorre el árbol en orden **infijo** y muestra los nodos en el orden en que se procesan.

b) Recorre el árbol en orden **postfijo** y muestra los nodos en el orden en que se procesan.

c) Inserta los siguientes valores en un **Árbol Binario de Búsqueda**: 4,7,3,6,2,9,8,5.

© Fernando Barber y Universitat de Valencia. Prohibida la copia o reproducción total o parcial sin previo consentimiento.

6. (1.0 ptos) Sabiendo que en C++ los montículos se pueden declarar de la siguiente manera, implementa un algoritmo de ordenación *heapsort* en C++ usando montículos.

```
priority_queue<T> cp_max;     // Motículo de máximos
priority_queue<int, vector<int>, std::greater<int> > cp_min;  // Mont. mínimos

vector<int> heapsort(const vector<int> & v)
{
```

```
}
```

© Fernando Barber y Universitat de Valencia. Prohibida la copia o reproducción total o parcial sin previo consentimiento.

7. (1.0 ptos) Dado el siguiente grafo.

	1	2	3	4	5	6	7
1	0	0	4	0	2	1	0
2	2	0	2	0	3	0	0
3	0	0	0	0	0	5	1
4	2	0	1	0	0	0	0
5	0	4	2	0	0	1	0
6	3	1	1	2	0	0	0
7	2	0	0	0	0	1	0

d) Realiza una **búsqueda en profundidad** del grafo empezando en el nodo 2. Dibuja el árbol que representa el recorrido e indica el orden en que se visitarían los nodos.

Árbol:

Nodos:

e) Realiza una **búsqueda en anchura** del grafo empezando en el nodo 2. Dibuja el árbol que representa el recorrido e indica el orden en que se visitarían los nodos.

Árbol:

Nodos:

8. (1.0 ptos) Dado el algoritmo de Dijkstra para matriz de adyacencia, del cual se muestra el comienzo a continuación, indicar qué valores contendrán los vectores *vecino* y *pvecino* después de ejecutarlo sobre el grafo representado por la matriz *p* y el nodo *v=1*. Indica su valor después de la inicialización y después de cada iteración del bucle principal.

```
ALGORITMO DIJKSTRA      // Matriz de adyacencia //
DATOS:      G = (N,A,p) // Grafo dirigido con pesos. p matriz de adyacencia
            V: Nat      // Nodo de inicio de los caminos
RESULTADO:  vecino: vector[1..n] de Nat //Mejores arcos factibles
            pvecino: vector[1..n] de Real // Peso de cada camino.
AUX:        B: vector[1..n] de Bool // Cjto de nodos
```

```
// Inicialización
Para i ← 1 hasta n hacer
  B[i] ← falso
  vecino[i] ← v
  pvecino[i] ← p[v,i]
fPara

B[v] ← cierto
// Bucle principal
Para i ← 2 hasta n hacer
...
fPara
```

Matriz *p*: El guión (-) representa que no hay arco, es decir, valor infinito.

	1	2	3	4	5
1	0	8	11	3	-
2	8	0	4	1	3
3	11	4	0	10	7
4	3	1	10	0	5
5	-	3	7	5	0

Inicialización

	1	2	3	4	5
vecino:					
pvecino:					

i=2:

	1	2	3	4	5
vecino:					
pvecino:					

i=3:

	1	2	3	4	5
vecino:					
pvecino:					

i=4:

	1	2	3	4	5
vecino:					
pvecino:					

i=5:

	1	2	3	4	5
vecino:					
pvecino:					

Indica la longitud del camino del nodo 1 al nodo 3:

© Fernando Barber y Universitat de Valencia. Prohibida la copia o reproducción total o parcial sin previo consentimiento.

9. (1.25 ptos) El siguiente algoritmo resuelve el problema del viajante (circuito hamiltoniano de menor peso) utilizando **backtracking con poda**. Complétalo. El grafo *g* contiene los tiempos para ir de un nodo a otro cualquiera. En el vector *camino* se devuelve el mejor camino encontrado, y en *mejor*, el tiempo completo del camino, desde el nodo de inicio 1 hasta volver otra vez al 1.

```
ALGORITMO TSP (i:Nat; nodo: Nat; suma: Real)
DATOS:      g: Vector[1..n][1..n] de Real // matriz de adyacencia
RES/GLOBAL: camino: vector[1..n] de Nat
            mejor: Real;
GLOBAL:     visit: vector[1..n] de Bool;
            v: vector[1..n] de Nat;
INICIO: mejor ← INF
        v[1] ← 1   // primera llamada TSP(1,1,0)
        visit inicializado a false, excepto visit[1] ← true
METODO:
Si i=n entonces
  si [                                    ] entonces
[                                                    ]
  fSi
sino
  para j ← 2 hasta n
    si [                                    ] //factible y poda
      visit[j] ← true;
      [                                    ]
      TSP( [                                    ] )
      visit[j] ← false;
  fpara
fSi
```